

Aula 01: Técnicas Clássicas de NLP

Estruturando um projeto de dados e construindo as primeiras representações de texto

Profa. Deisy Morselli Gysi, Ph.D.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Agenda

- Parte 1 — Projetos
- Parte 2: O problema
- Parte 3 — Dados textuais e objetivo analítico
- NLP clássico: fundamentos
- Tokenização e primeiras representações
- Sentimento e entidades
- Limitação da representação lexical

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Introdução

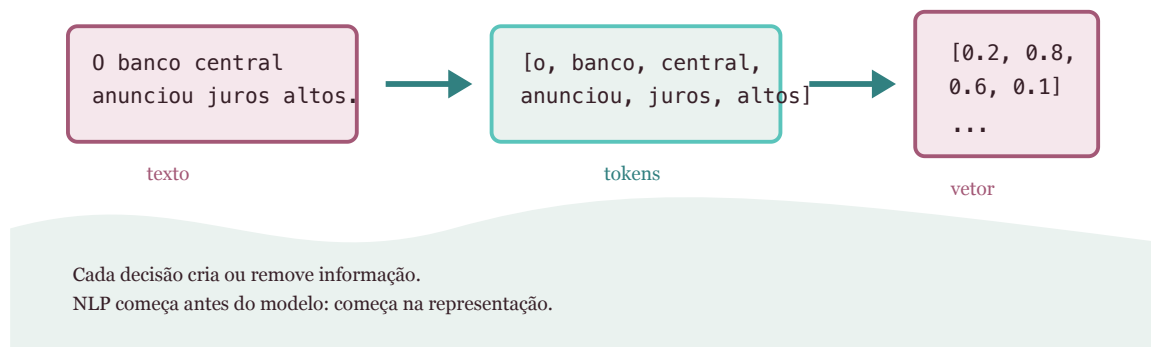
Antes de começarmos a falar em qualquer técnica de NLP precisamos entender como estruturamos um **projeto de análise de dados assistido por IA**:

Isto é: com pergunta de negócio, escopo e dados claros.

E só então transformamos texto em representações analisáveis e interpretáveis.

Uma pergunta de NLP

Como o computador transforma linguagem em evidência?



ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Estrutura da Aula

1. **Projeto e problema de negócio:** o que é um projeto de análise assistida por IA e como estruturá-lo.
2. **Dados textuais e objetivo analítico:** o que conta como dado e como delimitar a pergunta.
3. **NLP clássico:** tokenização, bag-of-words, TF-IDF, similaridade, temas, sentimento.
4. **Limitação da representação lexical:** onde contar palavras deixa de ser suficiente.
5. **Ponte:** Como fazemos a troca vetores esparsos por embeddings contextuais.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Parte 1 — Projetos

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



O analista de dados na era da IA generativa

Ferramentas de IA generativa hoje conseguem ler planilhas, textos e documentos, escrever código e devolver análises quase prontas em segundos. Então, para que serve o analista de dados?

A resposta curta: o trabalho não desaparece, ele se modifica. Continua sendo do analista **formular a pergunta certa, validar a resposta e decidir o que fazer com ela**: tanto o escopo, a qualidade dos dados e interpretação crítica do resultado são coisas que **nenhuma ferramenta decide sozinha**.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Janela de contexto: o que a IA consegue ver

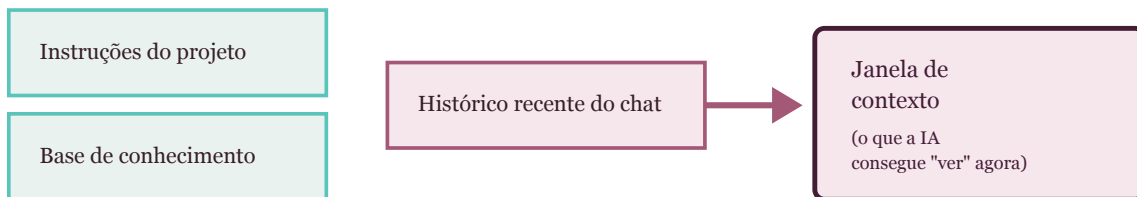


ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Janela de contexto: o que a IA consegue ver

O que entra na janela de contexto



O que fica fora da janela é esquecido ou precisa ser lembrado de novo.

- A IA só “enxerga” o que cabe na sua **janela de contexto**: instruções, documentos anexados e histórico recente da conversa.
- O que fica de fora da janela é esquecido — precisa ser reintroduzido ou reorganizado em um projeto persistente.
- Entender esse limite muda como organizamos dados e instruções ao longo de toda a especialização.
- Podemos resolver isso criando uma estrutura de “projeto”.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



O que é um “projeto” de análise assistida por IA

- Um **projeto** é um ambiente de trabalho autocontido: **mantém memória, histórico de interações e um contexto próprio ao longo do tempo**.
- Permite anexar bases de conhecimento (documentos, planilhas, materiais de referência).
- Permite **fixar instruções personalizadas de tom, estilo e nível técnico, que valem para toda conversa dentro do projeto**.
- Diferente de um chat avulso, o projeto acumula contexto e aprendizado à medida que é usado.



Quando vale a pena estruturar um projeto

Um projeto compensa o esforço de configuração quando:

- a tarefa é **recorrente ou contínua**, não uma pergunta única;
- é necessário **manter contexto** ao longo de dias ou semanas;
- há **documentos ou dados próprios** que devem informar todas as respostas;
- se deseja **padronizar** tom, formato e comportamento da IA;
- **várias pessoas** compartilham o mesmo fluxo de trabalho.



Como criar um projeto, em linhas gerais

Decidido que vale a pena estruturar um projeto, o processo de criação segue quatro passos, que detalharemos nos próximos slides:

1. **Criar:** dar nome e descrição claros, alinhados ao objetivo de negócio.
2. **Instruir:** fixar tom, estilo e regras de comportamento para todas as conversas do projeto.
3. **Alimentar:** anexar a base de conhecimento — documentos, planilhas e materiais de referência.
4. **Compartilhar:** definir quem tem acesso ao projeto e com qual nível de permissão.



Componentes do Projeto

Um projeto de análise assistida por IA reúne quatro peças



As quatro peças evoluem juntas: mais uso acumula contexto e aprendizado.



Componente 1 — Memória e histórico

- Armazena **interações anteriores dentro do escopo do projeto**.
- Mantém continuidade: a IA não repete perguntas já respondidas nem perde decisões já tomadas.
- É o componente que diferencia um **projeto de uma sequência de chats isolados**.



Componente 2 — Instruções do projeto

- **Definem tom, estilo e nível técnico esperado nas respostas.**
- **Garantem consistência:** todo membro do time recebe respostas no mesmo padrão.
- Funcionam como um “contrato de comportamento” que vale para qualquer pergunta feita dentro do projeto.



Componente 3 — Base de conhecimento

- Reúne documentos, planilhas e materiais relevantes para o projeto.
- Enriquece as respostas da IA com informação específica do domínio, não apenas conhecimento genérico.
- Qualidade da base de conhecimento é qualidade da análise: **dado desatualizado produz resposta desatualizada.**

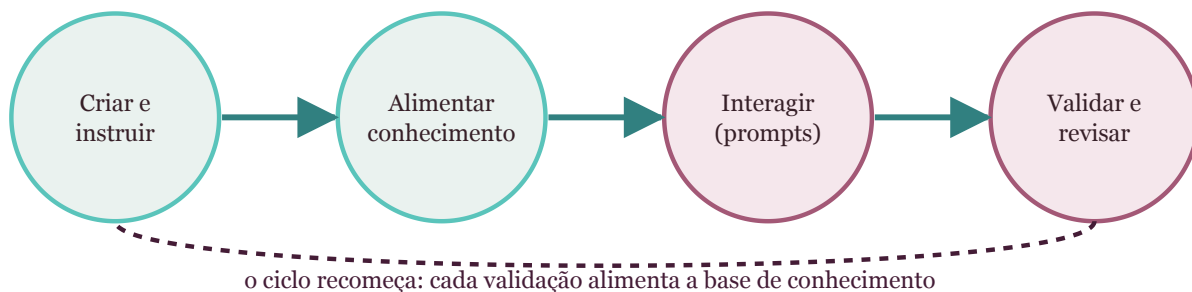


Componente 4 — Interações (prompts)

- É a comunicação contínua entre analista e IA dentro do projeto.
- Cada **prompt refina progressivamente o resultado anterior** — a análise raramente sai perfeita na primeira tentativa.
- Prompts claros e específicos reduzem retrabalho; prompts ambíguos multiplicam iterações.



Estruturando um projeto



ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Estrutura

Passo 1 — Criar o projeto

- Nome e descrição claros do objetivo.
- Regras de visibilidade: quem pode acessar.

Passo 2 — Instruções do projeto

- Contexto do domínio e do público-alvo.
- Tom, estilo e requisitos específicos de formato.

Passo 3 — Base de conhecimento

- Upload de documentos: texto (PDF, DOCX, CSV, TXT, HTML) e imagem (JPG, PNG).
- Nomeie arquivos de forma descritiva — o nome é usado para referenciar o documento nas conversas.

Passo 4 — Acesso ao projeto

- Defina níveis de permissão: ver, editar ou administrar.
- Compartilhamento entre pessoas da mesma equipe, quando a plataforma permitir.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Boas práticas para começar bem

- Comece com um **caso de uso específico**; evite um projeto genérico para “tudo”.
- Expanda o escopo apenas conforme a necessidade real aparecer.
- Um projeto bem delimitado é mais fácil de manter, auditar e explicar.



Boas práticas de organização e qualidade

- Agrupe documentos relacionados e facilite conexões entre fontes.
- Organize o projeto em torno de **um objetivo claro**, não de um tema vago.
- Mantenha a base de conhecimento atualizada; revise documentos periodicamente.
- Escreva instruções **sem ambiguidade** e **referencie documentos** pelo nome ao interagir.



Projeto x skill: qual a diferença?

	Projeto	Skill
O que é	Ambiente de trabalho persistente: memória, instruções, base de conhecimento	Um procedimento específico e reutilizável para uma tarefa concreta
Escopo	Amplo e contínuo — organiza um fluxo de trabalho inteiro	Estreito e pontual — resolve um tipo de tarefa bem definida
Persistência	Acumula contexto conversa após conversa	Não acumula histórico; é acionada sob demanda quando a tarefa se encaixa
Papel	O “ambiente” onde o trabalho acontece	Uma “ferramenta”/manual de procedimento usada dentro de um projeto



Tipos de análise: descritiva, diagnóstica, preditiva, prescritiva



Tipo	Pergunta central
Descritiva	O que aconteceu?
Diagnóstica	Por que aconteceu?
Preditiva	O que vai acontecer?
Prescritiva	O que devemos fazer?



Como a IA processa uma análise de dados

Dentro de um projeto de análise assistida por IA, o fluxo típico é:

1. Ler os dados enviados.
2. Entender a estrutura (colunas, tipos, tamanho).
3. Escrever código automaticamente (em geral Python) para a análise pedida.
4. Executar o código.
5. Retornar resultados e uma explicação do que foi feito.



Formatos de dados e boas práticas de entrada

Formatos comuns: Excel (.xlsx), CSV, JSON, PDF: dados estruturados funcionam melhor que dados soltos em texto corrido.

Faça isso: nomes de coluna claros; uma linha por registro; dados organizados em tabela; linguagem simples.

Evite isso: múltiplas tabelas no mesmo arquivo; linhas ou colunas vazias; abreviações confusas; **informação importante escondida em imagens.**



Transparência, validação e limites do projeto assistido por IA

- **Valide resultados críticos:** revise a lógica da análise antes de usá-la para decidir algo importante.
- **Não confie cegamente:** exporte o código gerado e avalie-o criticamente.
- **Limites:** a qualidade da resposta depende da qualidade dos dados; a IA pode interpretar mal o contexto do negócio; ela não substitui um especialista do domínio.
- Você ainda precisa julgar se a análise **faz sentido** para o seu problema.



Perguntas-chave para abrir um projeto de análise

Antes de criar qualquer projeto de análise assistida por IA, responda:

1. Quais dados eu já tenho hoje?
2. O que eu gostaria de entender melhor?
3. O que hoje demora muito tempo para analisar manualmente?

Comece simples, evolua rápido — essa é a lógica que vamos aplicar ao problema de negócio desta aula.



Parte 2: O problema

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Do projeto genérico ao nosso problema de negócio

Aplicamos o roteiro da Parte 1 a um caso concreto: uma redação precisa **monitorar grandes volumes de notícias** e decidir rapidamente o que merece checagem e publicação.

- Dados que já temos: manchetes e textos curtos, em mais de um idioma.
- O que queremos entender: tema, tom e possíveis duplicatas antes de qualquer publicação.
- O que hoje demora: leitura manual de centenas de textos por turno.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Minicase: monitoramento de notícias em 24 horas

Imagine uma redação recebendo centenas de textos por dia, em português e inglês, sobre economia, esporte, ciência e política. Antes de publicar qualquer nota, a equipe de plantão precisa:

- identificar do que trata cada texto;
- perceber se o tom é predominantemente positivo, negativo ou neutro;
- encontrar textos duplicados ou quase idênticos vindos de fontes diferentes.

Ninguém consegue ler tudo isso a tempo, por isso que precisamos de sinais textuais rápidos e auditáveis.



Pergunta operacional do minicase

Pergunta de negócio: que sinais textuais simples e auditáveis ajudam a priorizar o que revisar primeiro, sem exigir leitura integral de cada texto?

Essa pergunta orienta todas as técnicas da aula: **cada método será avaliado pela sua capacidade de responder a ela, não pela sua sofisticação matemática.**



Onde esse tipo de projeto se aplica

O mesmo roteiro (projeto → dados → objetivo → análise) aparece em:

- relatórios com estatísticas descritivas de conteúdo;
- análise financeira de notícias e relatórios;
- monitoramento de marketing e campanhas;
- triagem operacional de grandes volumes de texto — nosso minicaso de hoje.



Definições das Metodologias Básicas de NLP

- Natural Language Processing (NLP) clássico é o conjunto de técnicas que **representam texto por contagem, coocorrência e regras**, sem aprender representações a partir de grandes redes neurais.
- Toda afirmação sobre o corpus é checada com exemplos reais do fixture, não apenas com métricas agregadas.
- Duas fontes monolíngues (PT-BR e EN) são comparadas como *slices* de análise — não como um corpus bilíngue traduzido.



Parte 3 — Dados textuais e objetivo analítico

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



34

O que conta como dado textual neste curso

- Um dado textual é qualquer texto identificado por um `id`, com metadados mínimos (`idioma`, `categoria`, `fonte`) que permitam auditoria.
- Um título sem metadados não é um dado analisável — é apenas texto solto.
- Nesta aula usamos `data/fixtures/bilingual_news.csv`: notícias curtas em PT-BR e EN, já com esses metadados.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Definindo um objetivo analítico

Um objetivo analítico bem definido responde a três perguntas:

1. **Pergunta:** o que exatamente queremos saber sobre o texto?

- ex.: “quais notícias têm maior prioridade de checagem?”

2. **População:** quais documentos e período estão dentro do escopo?

- sobre o que posso falar com confiança, e o que está fora do escopo?

3. **Decisão:** que ação muda dependendo da resposta?

- o que vamos fazer com a informação obtida?

Para o minicaso: *“quais notícias recebidas hoje, em qualquer um dos dois idiomas, têm maior prioridade de checagem?”*



Ficha de dados (data card)

- Cada conjunto de dados do curso tem uma *data card* (`*_data_card.md`) documentando origem, tamanho e limitações.
- É um dicionário das variáveis: Contém as informações esperadas para o conjunto de dados.
- *Sempre* leia e entenda o conjunto de dados *antes* de qualquer análise.



Dados: o que temos e o que não temos

- Temos: textos curtos, rotulados por categoria e idioma, em volume pequeno e controlado.
- Não temos: volume realista de produção, ruído de digitação em escala, nem rótulos de “prioridade de checagem” prontos.
- Nosso objetivo hoje é entender a metodologia e não trabalhar com a escalabilidade.



Formatos e organização de dados textuais

Aplicando ao texto as mesmas boas práticas do Módulo de projetos:

- É importante termos uma linha por documento; colunas com nomes claros (`id`, `language`, `title`, `text`, `category`, `source`); texto em UTF-8.
- Não tenha múltiplos textos numa única célula; categorias inconsistentes ("`Sports`" vs "`sports`"); textos sem identificador único.



Qualidade de dados textuais: o que evitar

- IDs vazios ou duplicados invalidam qualquer contagem de corpus.
- Mistura de idiomas dentro do mesmo campo de texto sem marcação dificulta a interpretação de qualquer métrica.
- Textos truncados silenciosamente mudam o vocabulário sem aviso.



Ética e limites de uso de dados textuais

- Textos podem conter opiniões, nomes e dados sensíveis; trate-os com o mesmo cuidado de qualquer dado pessoal.
- Um score de sentimento ou uma entidade extraída **não é fato objetivo**, é uma inferência sujeita a erro e viés do método.
- Documente sempre a fonte e o escopo de coleta do dado textual analisado.



Do objetivo analítico à escolha de representação

Depois de fixar pergunta, população e decisão, a próxima escolha é: **que representação matemática do texto permite responder a essa pergunta?**

- Se a pergunta é sobre tema/tópico → contagem e coocorrência de termos.
- Se a pergunta é sobre similaridade entre textos → vetores comparáveis por uma métrica.
- Se a pergunta é sobre tom → léxico de polaridade.



NLP clássico: fundamentos



O que é NLP clássico

- Conjunto de técnicas que representam texto por **contagem, coocorrência e regras**, *sem aprender representações* a partir de grandes redes neurais.
- Precede os métodos baseados em *embeddings* e modelos de linguagem.
- Vantagem central: cada número do vetor final tem explicação auditável em termos do vocabulário original. Isto é, em casos onde a utilização de técnicas de NN não permite encontrar a fonte da informação, utilizar o NLP é a melhor alternativa.



O que a representação preserva e o que ela perde

- Toda representação de texto é uma escolha: **preserva algumas propriedades linguísticas e descarta outras deliberadamente**.
- Bag-of-words preserva frequência, mas descarta ordem.
- N-gramas preservam uma janela local de ordem, mas aumentam esparsidade.
- Nenhuma dessas representações preserva significado composicional completo.



NLP começa antes da predição



A predição (classificação, por exemplo) é a última etapa de uma cadeia de decisões de representação, e não o ponto de partida da análise.



Formalização da representação

Toda técnica desta aula é uma função ϕ que mapeia um documento d em um vetor:

$$\phi : d \mapsto \mathbf{x}_d \in \mathbb{R}^V$$

onde V é o tamanho do vocabulário (ou do espaço latente, quando reduzido). A pergunta metodológica de cada método é: **o que** ϕ preserva e o que descarta?



Pipeline de análise linguística



Cada etapa do pipeline é auditável: podemos sempre voltar do vetor final até o texto original que o gerou.



Tokenização e primeiras representações



Método — Tokenização



A unidade escolhida define tudo que pode ser contado depois.

Tudo começa cortando o texto em pedaços menores. A tokenização divide uma sequência de caracteres em unidades discretas (*tokens*) seguindo regras de fronteira: espaços, pontuação, ou até regras morfológicas mais finas para línguas aglutinantes. Formalmente, dado um texto $x = c_1 \dots c_m$, o tokenizador produz uma sequência $T(x) = (t_1, \dots, t_L)$.

Na prática: "0 banco central." vira [0] [banco] [central] [.]. Parece trivial, mas cada decisão de tokenização, ex. manter a pontuação? separar **microcrédito** em duas partes?, muda o vocabulário que vamos usar dali para frente.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Tokenização em Python

```
1 import re
2
3 text = "0 banco central."
4 tokens = re.findall(r"\w+|[\^\\w\\s]", text, flags=re.UNICODE)
5 print(tokens)
```

```
['0', 'banco', 'central', '.']
```

Só com a biblioteca padrão `re` já conseguimos reproduzir a regra de fronteira: letras/números viram um token, e cada sinal de pontuação vira outro.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Tokenização não é compreensão

Texto	Possível sequência de tokens
banco central	banco, central
juros altos	juros, altos
não aumenta	não, aumenta
microcrédito	uma palavra ou vários sub-tokens

- Remover a palavra **não** pode apagar informação essencial de sentimento.
- A mesma regra de tokenização se comporta de modo diferente em português e inglês. Pois, por exemplo, **microcrédito** é uma palavra única em português, mas **micro credit** são dois tokens em inglês.



O que é um n-grama?

Um **n-grama** é uma sequência contígua de n tokens. Para **o banco central anunciou juros altos**:

Tipo	Exemplos
Unigrama ($n = 1$)	banco, central, juros
Bigrama ($n = 2$)	banco central, juros altos
Trigrama ($n = 3$)	o banco central, central anunciou juros

O n-grama preserva contexto local e ordem, mas aumenta vocabulário e esparsidade.

- Como regra geral, n-gramas maiores capturam mais contexto, mas são mais raros e esparsos.



Método — N-gramas

janela de tamanho n

política monetária taxa de juros

bigrama: política monetária | taxa de juros

Formalmente, um n -grama é uma janela deslizando sobre a sequência de tokens: $G_n(d) = \{(t_i, t_{i+1}, \dots, t_{i+n-1})\}$. Na frase "O comitê de política monetária manteve a taxa de juros.", isso gera o bigrama política monetária e o trigramma taxa de juros — unidades que somem quando olhamos só para palavras isoladas, uma de cada vez.



Por que n-gramas importam — e onde param

- O bigrama taxa de juros preserva uma unidade local que unigramas isolados não capturam.
- Ainda assim, n -gramas não resolvem ironia, dependências longas ou sinonímia.
- Aumentar n aumenta contexto local, mas também aumenta drasticamente a esparsidade do vetor.



Antes de contar: da frase ao vetor

Com vocabulário [banco, central, juros, altos]:

```
1 "banco central juros" -> [1, 1, 1, 0]
2 "juros altos"          -> [0, 0, 1, 1]
```

Cada posição do vetor tem significado fixado pelo vocabulário; a ordem original das palavras desaparece.



Definindo n-gramas em Python

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 sentence = "O comitê de política monetária manteve a taxa de juros."
4
5 bigramas = CountVectorizer(ngram_range=(2, 2), token_pattern=r"(?u)\
6 bigramas.fit([sentence])
7 print(sorted(bigramas.vocabulary_))
8
9 trigramas = CountVectorizer(ngram_range=(3, 3), token_pattern=r"(?u)\
10 trigramas.fit([sentence])
11 print(sorted(trigramas.vocabulary_))
```

```
['a taxa', 'comitê de', 'de juros', 'de política', 'manteve a',
'monetária manteve', 'o comitê', 'política monetária', 'taxa de']
['a taxa de', 'comitê de política', 'de política monetária', 'manteve a
taxa', 'monetária manteve a', 'o comitê de', 'política monetária
manteve', 'taxa de juros']
```



Método — Bag-of-Words (BoW)

vocabulário

[banco, central, juro, alto]

[1, 1, 1, 0]

Contagem lexical, sem contexto global.

ordem perdida

banco central = central banco

O bag-of-words é a representação mais simples da aula: cada documento vira um vetor de contagens, uma posição por palavra do vocabulário, $\mathbf{x}_d = [c(t_1, d), c(t_2, d), \dots, c(t_V, d)]$ — e a ordem das palavras é jogada fora de propósito. É por isso que **banco central** e **central banco** produzem exatamente o mesmo vetor: para o bag-of-words, essas duas frases são indistinguíveis.



Exercício rápido — Bag-of-Words

Com o vocabulário [juro, gol, missão, governo], construa manualmente o vetor BoW de:

1. "o time marcou um gol e comemorou"
2. "o governo anunciou juro mais altos"

Depois confira: quantas posições do vetor ficaram em zero? O que isso revela sobre esparsidade em vocabulários maiores?



Bag-of-Words em Python

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 corpus = ["banco central", "central banco"]
4
5 vectorizer = CountVectorizer()
6 bow = vectorizer.fit_transform(corpus)
7
8 print(vectorizer.get_feature_names_out())
9 print(bow.toarray())
```

```
['banco' 'central']
[[1 1]
 [1 1]]
```

Mesmo vocabulário, mesmas contagens: o bag-of-words não enxerga a diferença de ordem entre as duas frases.



Método — TF-IDF

peso no documento

termo comum



baixo

termo raro



alto

Importância local × raridade no corpus.

TF-IDF (Term Frequency — Inverse Document Frequency) é uma forma de reponderar o bag-of-words para dar mais peso a termos raros e menos peso a termos frequentes. A ideia é que palavras muito comuns em todo o corpus carregam pouco poder discriminativo, enquanto palavras raras podem indicar tópicos específicos.

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log \left(\frac{N + 1}{1 + \text{df}(t)} \right) + 1$$

O efeito prático: **juros** pesa pouco num corpus cheio de notícias financeiras (todo mundo usa essa palavra), enquanto um termo raro como **telescópio** ganha um peso bem maior, porque aparece em poucos documentos e carrega mais sinal discriminativo.



Por que reponderar termos

- Termos muito frequentes em todo o corpus (ex.: `disse`, `hoje`) carregam pouco poder discriminativo.
- TF-IDF melhora a discriminação lexical entre documentos, mas **não cria significado contextual** — continua sendo contagem reponderada.
- A escolha de `min_df`/`max_df` muda o vocabulário efetivo e deve ser documentada como parte do método.



Exercício rápido — TF-IDF

Compare dois termos do fixture: um que aparece em quase todas as categorias (ex.: `hoje`) e um que aparece só na categoria `science` (ex.: `telescópio`). Qual peso TF-IDF você espera para cada um, e por quê?



TF-IDF em Python

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 corpus = [
4     "o banco central anunciou juros mais altos",
5     "os juros do banco central seguem em alta",
6     "o telescópio espacial captou uma nova imagem do planeta",
7 ]
8
9 vectorizer = TfidfVectorizer()
10 vectorizer.fit(corpus)
11

```

1.288 1.693

juros aparece em 2 dos 3 documentos (peso menor); **telescópio** aparece em só 1 (peso maior) — exatamente o efeito descrito acima.



Antes de medir: o que é “texto parecido”?

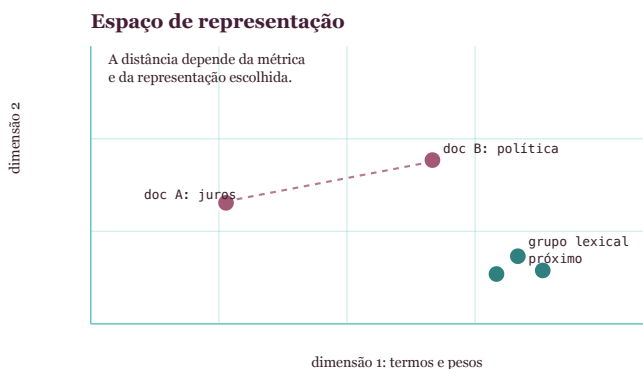
Uma métrica de similaridade transforma dois vetores em um número. Esse número depende inteiramente da representação escolhida, não existe uma medida universal de “mesmo significado”.

```

1 A: banco central anunciou juros
2 B: central anunciou juros altos

```

Podemos medir proximidade lexical entre A e B, mas não afirmar que os dois textos têm sentido idêntico.



Similaridade de cosseno

O cosseno compara o ângulo entre vetores e ignora sua magnitude absoluta:

$$\text{sim}_{\cos}(x, y) = \frac{x \cdot y}{\|x\| \|y\|}$$

- 1: mesma direção; 0: vetores ortogonais.
- Adequada para TF-IDF: documentos longos não ganham similaridade só por terem mais palavras.
- Mede sobreposição ponderada de termos, não semântica contextual.



Similaridade de cosseno em Python

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 doc_a = "banco central anunciou juros"
5 doc_b = "central anunciou juros altos"
6
7 vectorizer = TfidfVectorizer()
8 vectors = vectorizer.fit_transform([doc_a, doc_b])
9
10 sim = cosine_similarity(vectors[0], vectors[1])
11 print(round(sim[0, 0], 3))
```

0.603

Mesmo sem nenhuma palavra idêntica na mesma posição, A e B compartilham **central**, **anunciou** e **juros** — por isso o cosseno fica bem acima de zero.



Jaccard e distância Euclidiana

Jaccard compara conjuntos de tokens:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Intuitiva para presença/ausência, mas ignora frequência.

Euclidiana mede o comprimento da diferença:

$$d(x, y) = \sqrt{\sum_j (x_j - y_j)^2}$$

Sensível à escala e ao tamanho dos vetores.



Jaccard em Python

```
1 doc_a = "banco central anunciou juros"
2 doc_b = "central anunciou juros altos"
3
4 tokens_a = set(doc_a.split())
5 tokens_b = set(doc_b.split())
6
7 jaccard = len(tokens_a & tokens_b) / len(tokens_a | tokens_b)
8 print(round(jaccard, 3))
```

0.6

Dos 5 tokens únicos entre A e B, 3 são compartilhados (central, anunciou, juros) — daí o 0.6.



Distância Euclidiana em Python

```

1 from sklearn.feature_extraction.text import CountVectorizer
2 from sklearn.metrics.pairwise import euclidean_distances
3
4 doc_a = "banco central anunciou juros"
5 doc_b = "central anunciou juros altos"
6
7 vectorizer = CountVectorizer()
8 vectors = vectorizer.fit_transform([doc_a, doc_b]).toarray()
9
10 dist = euclidean_distances(vectors[[0]], vectors[[1]])
11 print(round(dist[0, 0], 3))

```

1.414

Ao contrário do Jaccard, a Euclidiana reage à diferença de contagens em **banco** e **altos** — os dois tokens que aparecem em só um dos textos.



Escolha da métrica

Métrica	Usa	Ignora	Boa pergunta
Cosseno	direção e pesos	magnitude	quais documentos têm perfil lexical semelhante?
Jaccard	tokens presentes	frequência e peso	quantos tokens são compartilhados?
Euclidiana	distância no espaço	invariância de escala	quão longe estão dois vetores normalizados?

No notebook usaremos cosseno com TF-IDF, pois queremos comparar perfis lexicais relativos.



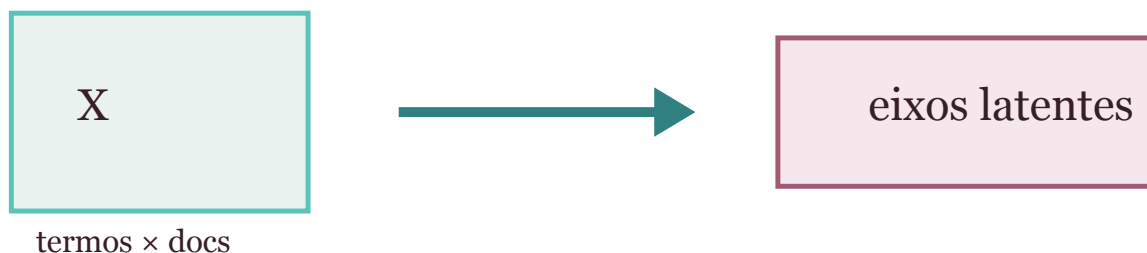
Exercício rápido — similaridade

Para os tokens `{banco, central, juro}` e `{central, juro, alto}`, calcule manualmente o coeficiente de Jaccard. Depois discuta: o resultado mudaria se um dos textos fosse duas vezes mais longo?



Método — LSA (Latent Semantic Analysis)

$$X \approx U_k \Sigma_k V_k^T$$



E se, em vez de olhar palavra por palavra, procurássemos padrões de coocorrência escondidos na matriz inteira? É essa a proposta do LSA: decompor a matriz termo-documento em eixos latentes, $X \approx U_k \Sigma_k V_k^T$, que capturam regularidades que nenhuma palavra isolada explicaria sozinha. No fixture, notícias com `telescópio`, `planeta` e `missão` tendem a se agrupar numa mesma componente “científica” — mesmo que nenhum desses três termos, isoladamente, defina o grupo.



LSA em Python

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.decomposition import TruncatedSVD
3
4 corpus = [
5     "o telescópio captou uma nova imagem do planeta",
6     "a missão espacial lançou um novo telescópio",
7     "o banco central anunciou juros mais altos",
8     "os juros do banco central seguem em alta",
9 ]
10
11 tfidf = TfidfVectorizer()

```

```

[[ 0.253  0.706]
 [ 0.076  0.738]
 [ 0.781 -0.228]
 [ 0.817 -0.069]]

```

O segundo eixo latente (2ª coluna) já separa ciência (valores positivos, perto de 0.7) de economia (valores negativos ou perto de zero) — mesmo sem nenhuma das duas palavras

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Método — K-Means

cada ponto → centróide mais próximo



grupo A



grupo B

O K-Means pega os vetores de documentos e os divide em k grupos, tentando minimizar a distância de cada ponto ao centro do seu grupo: $\arg \min_{C_1, \dots, C_k} \sum_j \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$. Um cuidado importante: o algoritmo só enxerga números, não significado — ele nunca vai dizer *por que* um grupo se formou. Antes de dar nome a um cluster, leia alguns documentos reais daquele grupo.



K-Means em Python

```

1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.cluster import KMeans
3
4 corpus = [
5     "o telescópio captou uma nova imagem do planeta",
6     "a missão espacial lançou um novo telescópio",
7     "o banco central anunciou juros mais altos",
8     "os juros do banco central seguem em alta",
9 ]
10
11 X = TfidfVectorizer().fit_transform(corpus)
12
13 [1 1 0 0]

```

Os dois primeiros documentos (ciência) caem no mesmo cluster, e os dois últimos (economia) caem no outro — sem que o algoritmo jamais tenha visto os rótulos `science/business`.



Exercício rápido — LSA e K-Means

Agrupe mentalmente as categorias do fixture (`business`, `sports`, `science`, `world`) em dois grupos usando apenas o vocabulário que você espera em cada uma. Depois confira no notebook se o K-Means encontrou a mesma separação.



Sentimento e entidades

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Método — Sentimento baseado em léxico

O time conquistou o título

conquistou + título +1 → score = +2

Transparente, mas sensível a negação, ironia e domínio.

A ideia aqui é bem direta: cada palavra recebe uma polaridade pré-definida em um léxico (positiva, negativa ou neutra), e o score do texto é simplesmente a soma ponderada dessas polaridades, $s(d) = \sum_{t \in d} w_t \cdot \text{pol}(t)$. No fixture, **conquistou** puxa o score para cima e **greve** puxa para baixo — um léxico funciona como uma calculadora de palavras-chave, nada mais sofisticado do que isso.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Limites do sentimento léxico

- Falha sistematicamente com **negação** (**não gostei**), **ironia** e mudança de domínio do léxico.
- O score é auditável termo a termo, mas isso não o torna correto, é apenas rastreável.
- Deve sempre ser avaliado contra uma fixture anotada manualmente, nunca aceito sem checagem.



Exercício rápido — sentimento

Calcule manualmente o score de sentimento de "o time não decepcionou e conquistou o título" usando um léxico simples de 4 palavras. O resultado muda se você ignorar a negação?



Sentimento léxico em Python

```
1 from LeIA import SentimentIntensityAnalyzer
2
3 analisador = SentimentIntensityAnalyzer()
4
5 frases = [
6     "o atendimento foi bom",
7     "o atendimento não foi bom",
8     "o atendimento foi ruim",
9     "o atendimento não foi ruim",
10 ]
11
```

o atendimento foi bom -> 0.421

o atendimento não foi bom -> 0.153

o atendimento foi ruim -> -0.557

o atendimento não foi ruim -> -0.7

O **LeIA** (léxico VADER adaptado para português) reconhece **bom** e **ruim** corretamente: a frase simples com **bom** fica positiva (0.42) e com **ruim** fica negativa (-0.56). Ao negar com **não**, "**não**

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



foi bom" cai para 0.15 (ainda positivo, mas atenuado, ok). Já "**não foi ruim**" deveria neutralizar ou até inverter o negativo, mas o score fica **ainda mais negativo** (-0.70) — mesmo um léxico maduro e amplamente usado tropeça sistematicamente na negação, confirmando o limite descrito no slide anterior.



Método — Extração de entidades (NER)

O Banco Central avaliou a inflação

Banco Central

ORG

inflação

CONCEITO

Reconhecimento de entidades nomeadas (NER) é a tarefa de encontrar, dentro do texto corrido, os trechos que se referem a pessoas, organizações, lugares, datas ou outras categorias de interesse. Formalmente, dada a sequência de tokens $t_1, \dots, t_n$, o modelo prevê uma sequência de tags $\hat{y} = \arg \max_y P(y \mid t_1, \dots, t_n)$ seguindo um esquema de marcação como o BIO. Na prática, é algo como marcar 0 [Banco Central/ORG] avaliou a [inflação/CONCEITO]. — o texto continua o mesmo, só ganha anotações por cima.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Utilidade e limite das entidades extraídas

- Extrair pessoa, organização, local e data pode acelerar bastante a triagem de um grande volume de texto.
- Erros comuns: fronteira errada da entidade e confusão entre categorias próximas.
- Extrair uma entidade **não implica** relação causal nem garantia de correção factual.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Exercício rápido — NER

No trecho "O Banco Central e o Ministério da Fazenda discutiram a nova taxa de juros", identifique manualmente as entidades candidatas e discuta uma fronteira ambígua.



NER em Python

```
1 import spacy
2
3 nlp = spacy.load("pt_core_news_sm")
4 texto = "O Banco Central e o Ministério da Fazenda discutiram a nova
5
6 doc = nlp(texto)
7 for ent in doc.ents:
8     print(ent.text, "->", ent.label_)
```

Banco Central -> LOC

Ministério da Fazenda -> LOC

O modelo encontra as duas instituições, mas rotula ambas como **LOC** (local) em vez de **ORG** (organização) — e “taxa de juros” corretamente não vira entidade nenhuma. É exatamente a “confusão entre categorias próximas” citada no slide anterior: mesmo um modelo real de NER erra a categoria de nomes de instituições, então a saída sempre precisa de revisão humana antes de virar dado confiável.



Limitação da representação lexical

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



O limite do baseline lexical

Vale parar e nomear o que fizemos até aqui: toda técnica desta aula é, no fundo, uma forma de **contar e reponderar palavras** — nenhuma delas entende significado composicional de verdade.

Isso tem um preço. Similaridade semântica entre textos com vocabulários diferentes (sinônimos, paráfrases) continua sendo um problema que a contagem simplesmente não resolve. E, pior: termos espúrios podem parecer estatisticamente fortes no nosso fixture pequeno, sem qualquer robustez fora dele.

ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL GENERATIVA



Onde a contagem de palavras falha

- 1 "o time não decepcionou" -> positivo (correto: elogio)
- 2 "o time decepcionou" -> negativo

Um modelo lexical ingênuo trata **não decepcionou** como duas palavras isoladas e pode errar o sinal completo se **não** for descartado no pré-processamento.



Sinônimos, negação e ironia: três armadilhas

Três situações simples bastam para derrubar qualquer método puramente lexical:

1. **Sinônimos:** "crescimento" e "expansão", cachorro e cão, casa e residência recebem vetores completamente diferentes, mesmo dizendo quase a mesma coisa.
2. **Negação:** "não subiu" e "subiu" compartilham quase todo o vocabulário, mas têm sentidos opostos — uma única palavra muda tudo.
3. **Ironia e contexto discursivo:** depende de informação que nenhuma contagem local de palavras consegue captar.



O que motiva ir além do léxico

- Precisamos de uma representação onde palavras de sentido próximo fiquem **próximas no espaço vetorial**, mesmo sem compartilhar a mesma grafia.
- Precisamos de uma representação sensível à **ordem e ao contexto** ao redor de cada palavra, não só à sua presença.
- Isso é exatamente o que embeddings contextuais prometem entregar.



De vetores esparsos a representações densas

	Representação lexical	Embeddings
Dimensão	Tamanho do vocabulário	Fixa e densa (ex.: 384, 768)
Zeros	Maioria das posições	Poucas ou nenhuma
Sinônimos	Vetores distantes	Vetores próximos
Ordem/contexto	Só em n-gramas locais	Considerada por toda a frase



O que são embeddings, em uma frase

Um **embedding** é uma função aprendida que mapeia cada palavra (ou frase) para um vetor denso, de forma que palavras usadas em contextos semelhantes recebam vetores próximos — sem precisarem compartilhar nenhuma letra em comum.



BERT e o papel do contexto

- BERT (Bidirectional Encoder Representations from Transformers) são modelos de aprendizado profundo para o processamento de linguagem natural. Criados pela Google em 2018.
- Modelos como BERT geram um vetor diferente para a **mesma palavra** dependendo da frase em que ela aparece.
- Isso resolve, ao menos parcialmente, o problema de polissemia que a contagem de palavras não consegue tratar.
- O custo: mais dados, mais computação e menos transparência direta sobre “por que” um vetor tem aquele valor.



Diferenças dos BERTs para os modelos clássicos

Hipóteses que testaremos, mantendo os mesmos splits e métricas desta aula:

1. Embeddings contextuais reduzem erros causados por sinonímia.
2. Modelos tipo BERT melhoram a robustez entre os dois idiomas em textos curtos.
3. O ganho de qualidade tem um custo computacional que precisa ser justificado pelo objetivo analítico.

TF-IDF → embeddings contextuais → engenharia de contexto para LLMs



Uma frase, um vetor: recapitulando os métodos

Método	Preserva	Descarta
Bag-of-words	frequência	ordem
N-gramas	ordem local	independência do restante da frase
TF-IDF	raridade discriminativa	significado contextual
Cosseno/Jaccard	sobreposição lexical	semântica composicional
LSA	coocorrência latente	interpretação garantida do eixo
Sentimento léxico	polaridade termo a termo	negação e ironia



LAB 1 — roteiro operacional

1. Carregar e auditar o dataset [data/Songs.csv](#).
2. Comparar contagem, TF-IDF e n-gramas.
3. Inspeccionar vocabulário, esparsidade e termos salientes por classe.
4. Calcular similaridade e observar quais documentos ficam próximos.

